Elementary Operations on Quasi-Toeplitz Matrices

PCCA6

Ali Arda BARUT Rafael LLAMAS

May 19, 2026

Context

- Operations on large structured matrices are often limited by memory bandwith.
- Standart operatons on n^2 elements lead to unnecessary computations.

Goal

Optimize memory footprint and execution time on structured matrices by implementing displacement operators and generator-based representations.

Context

Where do we see these structured matrices?

- Sylvester matrix
- Diagonal gradients
- Quantum channel calculations in classical quantum simulations
- Digital Signal Processing
- ...

Toeplitz Matrices

Definition

A matrix $A \in \mathbb{K}^{n \times m}$ is **Toeplitz** if $A_{i,j} = A_{i+1,j+1}$ for all valid i, j.

$$A = \begin{pmatrix} a_0 & a_{-1} & \cdots & \cdots & a_{-(m-1)} \\ a_1 & a_0 & \cdots & \cdots & a_{-(m-2)} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ a_{n-1} & \cdots & a_1 & a_0 & a_{m-n} \end{pmatrix}$$

Displacement

Displacement Operator

$$\nabla A = A - ZAZ^T = GH^T$$

For Toeplitz: $\operatorname{rank}(\nabla A) \leq 2$

Notice

Storing (G, H) instead of A reduces memory from $O(n^2)$ to $O(\alpha n)$.

Example - Displacement

Toeplitz Matrix A

$$A = \begin{pmatrix} 4 & 3 & 2 & 1 \\ 5 & 4 & 3 & 2 \\ 6 & 5 & 4 & 3 \\ 7 & 6 & 5 & 4 \end{pmatrix}$$

Displacement $\nabla A = A - ZAZ^T$

$$\nabla A = \begin{pmatrix} 4 & 3 & 2 & 1 \\ 5 & 0 & 0 & 0 \\ 6 & 0 & 0 & 0 \\ 7 & 0 & 0 & 0 \end{pmatrix}$$

Only first row and column stays. $\mathrm{rank} = 2$.

Example - Displacement

Displacement Matrix A

$$\nabla A = \begin{pmatrix} 4 & 3 & 2 & 1 \\ 5 & 0 & 0 & 0 \\ 6 & 0 & 0 & 0 \\ 7 & 0 & 0 & 0 \end{pmatrix}$$

Generators G and H

$$G = \begin{pmatrix} 4 & 0 \\ 5 & 1 \\ 6 & 0 \\ 7 & 0 \end{pmatrix} \qquad H^T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 3 & 2 & 1 \end{pmatrix}$$

Reconstruction

$$A = \sum_{k=1}^{\alpha} L(g_k)\, U(h_k)$$

Defining L and U are Lower and Upper matrices respectfully:

$$L(g_k) = \begin{bmatrix} g_1 & 0 & 0 & 0 \\ g_2 & g_1 & 0 & 0 \\ g_3 & g_2 & g_1 & 0 \\ g_4 & g_3 & g_2 & g_1 \end{bmatrix} \qquad U(h_k) = \begin{bmatrix} h_1 & h_2 & h_3 & h_4 \\ 0 & h_1 & h_2 & h_3 \\ 0 & 0 & h_1 & h_2 \\ 0 & 0 & 0 & h_1 \end{bmatrix}$$

Example - Reconstruction

$$A = L(g^{(1)})U(h^{(1)}) + L(g^{(2)})U(h^{(2)})$$

$$L(g^{(1)})U(h^{(1)}) = \begin{pmatrix} 4 & 0 & 0 & 0 \\ 5 & 4 & 0 & 0 \\ 6 & 5 & 4 & 0 \\ 7 & 6 & 5 & 4 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 4 & 0 & 0 & 0 \\ 5 & 4 & 0 & 0 \\ 6 & 5 & 4 & 0 \\ 7 & 6 & 5 & 4 \end{pmatrix}$$

$$L(g^{(2)})U(h^{(2)}) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 3 & 2 & 1 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 3 & 2 & 1 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Complexity - Reconstruction

Reconstruction

$$A = \sum_{k=1}^{\alpha} L(g_k)\, U(h_k)$$

Complexities

Naive: $O(n^3)$
Pointer Shifts: $O(n^2)$
[TODO: Optimized:]

Addition of Toeplitz-like Matrices

Generator Addition

$$A \simeq (T, U), B = (G, H) \Rightarrow A + B = (T|G, U|H)$$

Notice
from $O(n^2)$ to $O(\alpha M(n))$.

Multiplication of Toeplitz-like Matrices

Generator Multiplication with Vectors

blablabla

Notice

from $O(???)$ to $O(???)$.

Multiplication of Toeplitz-like Matrices

Generator Multiplication

$$A = (T, U), \qquad B \qquad = (G, H)$$
$$W = ZAZ^tG$$
$$V = B^tU$$
$$\alpha = ZA$$
$$\beta = ZB^t$$
$$C = (T|W|\alpha, V|H|\beta)$$

Notice

from $O(n^3)$ to $O(\alpha^2 M(n))$.

Compression

Operations like addition and multiplication forcefully increase the rang of the generators.

$$G = \begin{pmatrix} 4 & \cdots \\ 5 & \cdots \\ 6 & \cdots \\ 7 & \cdots \end{pmatrix} \qquad H^T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

Compression

Definition

Matrices $G \in \mathbb{R}^{n \times \alpha}$ and $H \in \mathbb{R}^{n \times \alpha}$ representing a displacement matrix $\nabla A = GH^T$, where the actual rank of ∇A is $r < \alpha$.

Goal: Find generators $G' \in \mathbb{R}^{n \times r}$ and $H' \in \mathbb{R}^{n \times r}$ s.t. the product is the same:

$$G'H'^T = GH^T$$

Compression

Algorithm 1 Generator Compression

function GR_MAT_GENERATOR_COMPRESS_AUX(G, H)
  $P, L, U \leftarrow$ LU_Decomposition(G^T) ▷ $PG^T = LU$
  $r \leftarrow$ Number of non-zero rows in U ▷ rank
  if $r < 1$ **then**
    return Generators of a 0 matrix
  end if
  $G_D \leftarrow (U_{[0:r-1],*})^T$
  $H_D \leftarrow HP^T \times L_{*,[0:r-1]}$
  return G_D, H_D
end function

Giving us a complexity of $O(\alpha^2 n)$

Inversion

Inverting a Toeplitz matrix represented by generators allows us to efficiently solve systems. But can also be used in:

- Image unblurring and de-echo.
- Speech compression and financial forecasting.

Inversion

Inverting a Toeplitz matrix represented by generators allows us to efficiently solve systems.

We can apply a divide-and-conquer algorithm, aiming to reduce the complexity from $O(n^3)$ to $O(n \log^2 n)$.

Inversion

Algorithm 2 Strassen-type Inversion for Generators: $\textsc{StrassenInv}(G_A, H_A)$

if $n = 1$ **then return** scalar inverse (G_D, H_D)
end if ▷ **Step 1: Base Case**
$G_a, H_a, \ldots, G_d, H_d \leftarrow \textsc{Split}(G_A, H_A)$ ▷ **Step 2: Quadrants**
$G_e, H_e \leftarrow \textsc{StrassenInv}(G_a, H_a)$ ▷ **Step 3: Recursion 1 ($e := a^{-1}$)**
$G_{ce}, H_{ce} \leftarrow \mathcal{C}(G_c, H_c \otimes G_e, H_e)$ ▷ **Step 4: Schur Complement**
$G_{eb}, H_{eb} \leftarrow \mathcal{C}(G_e, H_e \otimes G_b, H_b)$
$G_{ceb}, H_{ceb} \leftarrow \mathcal{C}(G_c, H_c \otimes G_{eb}, H_{eb} \times -1)$
$G_S, H_S \leftarrow \mathcal{C}(G_d, H_d \oplus G_{ceb}, H_{ceb})$ ▷ $S := d - ceb$
$G_t, H_t \leftarrow \textsc{StrassenInv}(G_S, H_S)$ ▷ **Step 5: Recursion 2 ($t := S^{-1}$)**
$G_{ebt}, H_{ebt} \leftarrow \mathcal{C}(G_{eb}, H_{eb} \otimes G_t, H_t)$ ▷ **Step 6: Strassen Assembly**
$G_{tce}, H_{tce} \leftarrow \mathcal{C}(G_t, H_t \otimes G_{ce}, H_{ce})$
$G_{ebtce}, H_{ebtce} \leftarrow \mathcal{C}(G_{ebt}, H_{ebt} \otimes G_{ce}, H_{ce})$
$G_x, H_x \leftarrow \mathcal{C}(G_e, H_e \oplus G_{ebtce}, H_{ebtce})$ ▷ $x := e + ebtce$
$G_y, H_y \leftarrow \textsc{Negate}(G_{ebt}, H_{ebt})$; $G_z, H_z \leftarrow \textsc{Negate}(G_{tce}, H_{tce})$
$G_{Inv}, H_{Inv} \leftarrow \textsc{Pack}(G_x, H_x, G_y, H_y, G_z, H_z, G_t, H_t)$ ▷ **Step 7: Pack**
return $\mathcal{C}(G_{Inv}, H_{Inv})$

Notation: $\mathcal{C}(\cdot)$ *= Compress,* $\otimes$ *= Multiply Generators,* $\oplus$ *= Add Generators*

Inversion

Algorithm 3 Strassen-type Inversion for Generators: $\textsc{StrassenInv}(G_A, H_A)$

if $n = 1$ **then return** scalar inverse (G_D, H_D)
end if
$G_a, H_a, \ldots, G_d, H_d \leftarrow \textsc{Split}(G_A, H_A)$ ▷ **Step 2: Quadrants**
$G_e, H_e \leftarrow \textsc{StrassenInv}(G_a, H_a)$ ▷ **Step 3: Recursion 1** ($e := a^{-1}$)
$G_{ce}, H_{ce} \leftarrow \mathcal{C}(G_c, H_c \otimes G_e, H_e)$ ▷ **Step 4: Schur Complement**
$G_{eb}, H_{eb} \leftarrow \mathcal{C}(G_e, H_e \otimes G_b, H_b)$
$G_{ceb}, H_{ceb} \leftarrow \mathcal{C}(G_c, H_c \otimes G_{eb}, H_{eb} \times -1)$
$G_S, H_S \leftarrow \mathcal{C}(G_d, H_d \oplus G_{ceb}, H_{ceb})$ ▷ $S := d - ceb$
$G_t, H_t \leftarrow \textsc{StrassenInv}(G_S, H_S)$ ▷ **Step 5: Recursion 2** ($t := S^{-1}$)
$G_{ebt}, H_{ebt} \leftarrow \mathcal{C}(G_{eb}, H_{eb} \otimes G_t, H_t)$ ▷ **Step 6: Strassen Assembly**
$G_{tce}, H_{tce} \leftarrow \mathcal{C}(G_t, H_t \otimes G_{ce}, H_{ce})$
$G_{ebtce}, H_{ebtce} \leftarrow \mathcal{C}(G_{ebt}, H_{ebt} \otimes G_{ce}, H_{ce})$
$G_x, H_x \leftarrow \mathcal{C}(G_e, H_e \oplus G_{ebtce}, H_{ebtce})$ ▷ $x := e + ebtce$
$G_y, H_y \leftarrow \textsc{Negate}(G_{ebt}, H_{ebt})$; $G_z, H_z \leftarrow \textsc{Negate}(G_{tce}, H_{tce})$
$G_{Inv}, H_{Inv} \leftarrow$ **Pack**$(G_x, H_x, G_y, H_y, G_z, H_z, G_t, H_t)$ ▷ Step 7: Pack
return $\mathcal{C}(G_{Inv}, H_{Inv})$

Notation: $\mathcal{C}(\cdot)$ = *Compress,* $\otimes$ = *Multiply Generators,* $\oplus$ = *Add Generators*

Live Benchmarking

Thank you